

Generalized Sudoku as a Boolean Satisfiability Problem: Comparing Native and Compact CNF encoding against Backtracking

Ashwot Acharya , Bishesh Bohora

September 9, 2026

Abstract

Sudoku is a well-known puzzle whose generalized version can be formulated as a decision and search problem over an $n^2 \times n^2$ grid. From a computational perspective, generalized Sudoku is NP-complete, and finding a valid completion corresponds to solving a function problem in FNP. This project presents a Boolean satisfiability (SAT) based approach for solving generalized Sudoku instances by reducing the puzzle to a propositional formula in Conjunctive Normal Form (CNF).

We formally encode the structural constraints of Sudoku—definedness and uniqueness across cells, rows, columns, and sub-grids—into Boolean variables of the form $x_{i,j,k}$, representing the assignment of digit k to cell (i, j) . The resulting constraints are translated into CNF and expressed in DIMACS format, enabling compatibility with standard SAT solvers. We adopt an optimized encoding strategy following Kwon and Jain [Kwon and Jain, 2006] and compare it against a naive (full) encoding, measuring variable and clause reduction across grid sizes from 4×4 to 36×36 .

The implementation generates CNF files using Python, maps logical variables to integer representations, and solves instances using the Satch SAT solver. We also compare the SAT-based method against a classical backtracking approach with MRV heuristics. A comprehensive benchmark of 75 runs across 5 grid sizes with 3 repetitions each demonstrates that SAT solving with compact encoding achieves up to $4.81\times$ speedup over backtracking on solve-only time for 36×36 grids, while the compact encoding reduces variables by up to 98.7% and clauses by up to 99.9% compared to naive encoding. Wilcoxon signed-rank and Friedman non-parametric tests confirm statistically significant performance differences at 4 of the 5 grid sizes ($p < 0.05$). All solutions achieve a 100% validation pass rate, verifying correctness of the SAT-based approach.

Contents

1	Introduction	5
1.1	Sudoku	5
1.1.1	Complexity of Generalized Sudoku Problem	5
1.1.2	Sudoku as CSP	5
1.2	The Boolean Satisfiability Problem	6
1.2.1	Complexity of SAT	6
1.2.2	SAT Solvers	6
1.3	Justification for SAT approach	6
2	Mathematical Formulation	7
2.1	Conjunctive Normal Form	7
2.2	Representation of Sudoku As Boolean Expression (CNF)	7
2.2.1	Definedness	7
2.2.2	Uniqueness	8
2.2.3	Final Formula	10
3	Implementation	11
3.1	Computer Representation	11
3.1.1	DIMACS Format	11
3.2	Mapping variables to integers	11
3.3	Solution Using SAT Solvers	11
3.4	Dual CNF Encoding	12
3.4.1	Naive (Full) Encoding ϕ	12
3.4.2	Compact Encoding ϕ'	12
4	Experimental Setup	12
4.1	Dataset	12
4.2	Benchmark Protocol	13
4.3	Metrics	13
5	Results	13
5.1	Solver Performance (Solve-Only Time)	13
5.2	End-to-End Performance (Encoding + Solve)	14
5.3	CNF Dual Encoding Comparison	15
5.4	Scaling Visualization	15
6	Statistical Analysis	15
6.1	Methodology	15
6.2	Wilcoxon Signed-Rank Test: BT vs. SAT Compact	18
6.3	Friedman Test: Multi-Solver Comparison	18
6.4	Solution Correctness	19

7	Discussion	19
7.1	Encoding Optimization is Critical	19
7.2	SAT vs. Backtracking: A Size-Dependent Trade-off	19
7.3	Encoding Time Dominates E2E	19
7.4	Statistical Rigor	20
7.5	Limitations	20
8	Conclusion	20
A	Compact Encoding (sudoku_to_cnf.py)	22
B	Naive Encoding (dual_encoder.py)	24
C	Solution Decoder	26
D	CDCL Solver Core (C)	27
E	Benchmark Orchestration (unified_benchmark.py)	27
F	Statistical Analysis (statistical_analysis.py)	28

List of Acronyms

SAT	Boolean Satisfiability Problem
CNF	Conjunctive Normal Form
CDCL	Conflict-Driven Clause Learning
DPLL	Davis–Putnam–Logemann–Loveland Algorithm
DIMACS	Center for Discrete Mathematics and Theoretical Computer Science
MRV	Minimum Remaining Values
VSIDS	Variable State Independent Decaying Sum
UNSAT	Unsatisfiable
SAT (result)	Satisfiable
BT	Backtracking
E2E	End-to-End

1 Introduction

1.1 Sudoku

A puzzle in which an $n^2 \times n^2$ grid consisting of n “ $n \times n$ ” sub-grids is to be filled with numbers from 1 to n^2 so that every row, column, and region contains only one instance of each number, $n \geq 1$. In a Sudoku puzzle, some cells are already filled; these are called **clues**. The **Sudoku Problem** is to determine whether there exists a completion such that the aforementioned conditions are satisfied.

The most common format for Sudoku is the 9×9 grid. In this project we work on larger variants. For an arbitrary positive integer n , we call it **Generalized Sudoku**.

1			4	8	9			6
7	3						4	
					1	2	9	5
		7	1	2		6		
5			7		3			8
		6		9	5	7		
9	1	4	6					
	2						3	7
8			5	1	2			4

1	5	2	4	8	9	3	7	6
7	3	9	2	5	6	8	4	1
4	6	8	3	7	1	2	9	5
3	8	7	1	2	4	6	5	9
5	9	1	7	6	3	4	2	8
2	4	6	8	9	5	7	1	3
9	1	4	6	3	7	5	8	2
6	2	5	9	4	8	1	3	7
8	7	3	5	1	2	9	6	4

Figure 1: 9×9 Sudoku and its solution [Arizona, 2024]

1.1.1 Complexity of Generalized Sudoku Problem

The Partial Latin Square Completion problem was shown to be NP-complete [Cobourn, 1984]. Yato and Seta [Yato, 2003] demonstrated that Partial Latin Square can be parsimoniously reduced to Number Place (generalized Sudoku), thereby establishing the ASP-completeness of Sudoku, which captures the computational hardness of finding another solution. Because ASP-completeness implies NP-completeness, generalized Sudoku is **NP-complete** [Yato, 2003]. Moreover, since the reduction in [Yato, 2003] is parsimonious (preserving the number of solutions), ASP-completeness also implies that counting the solutions of a generalized Sudoku instance is #P-complete.

1.1.2 Sudoku as CSP

Sudoku can naturally be formulated as a Constraint Satisfaction Problem (CSP), where each cell is a variable, the domain consists of the numbers $\{1, \dots, n^2\}$, and the constraints enforce row, column, and subgrid consistency. Classical approaches solve this CSP using backtracking search with heuristics such as Minimum Remaining Values (MRV) [Russell and Norvig, 2010, Dechter, 2003].

1.2 The Boolean Satisfiability Problem

In logic and computer science, the Boolean satisfiability problem (sometimes called propositional satisfiability problem and abbreviated SATISFIABILITY, SAT or B-SAT) asks whether there exists an interpretation that satisfies a given Boolean formula. In other words, it asks whether the formula’s variables can be consistently replaced by the values TRUE or FALSE to make the formula evaluate to TRUE. If this is the case, the formula is called satisfiable, else unsatisfiable [Wikipedia, 2024].

$a \vee \neg b$ is SATISFIABLE with $a = 1$ and $b = 0$.

$a \wedge \neg a$ is UNSATISFIABLE.

1.2.1 Complexity of SAT

SAT is NP-complete. In fact, it was the first problem to be known as such. As proved by Stephen Cook [Cook, 1971] at the University of Toronto in 1971 and independently by Leonid Levin at the Russian Academy of Sciences in 1973 [Wikipedia, 2024].

1.2.2 SAT Solvers

SAT solvers are computer programs or algorithms that check if there exist some assignment of Boolean variables in the given problem such that it evaluates to True. SAT solvers also provide the assignments. The DPLL algorithm [Davis et al., 1962] laid the foundation for modern search-based SAT solving. Conflict-Driven Clause Learning (CDCL), introduced by GRASP [Marques-Silva and Sakallah, 1996] and dramatically accelerated by Chaff’s VSIDS decision heuristic [Moskewicz et al., 2001], is now the standard architecture in solvers such as MiniSat [Eén and Sörensson, 2003] and the Satch solver used in this work [Biere, 2021]. CDCL combines unit propagation, conflict analysis, clause learning, and non-chronological backtracking to efficiently prune the search space. Cardinality constraints of the kind appearing in our Sudoku encoding are commonly handled with compact CNF encodings such as the sequential-counter encoding of Sinz [Sinz, 2005].

1.3 Justification for SAT approach

Since generalized Sudoku is NP-complete (and thus belongs to NP), it can be reduced in polynomial time to any other NP-complete problem, such as Boolean satisfiability (SAT). Note that the aforementioned complexity is for when n grows without bound, but in practice we have a finite grid. This reduction provides a formal justification for using SAT solvers to find Sudoku solutions: by converting a Sudoku instance into an equivalent SAT instance, we can leverage modern SAT-solving algorithms to efficiently compute a valid completed grid. This approach is theoretically sound, because solving the SAT instance is guaranteed to yield a solution to the original Sudoku problem whenever one exists. SAT solving also aids in efficient solving of CSPs [Lardeux et al., 2020].

2 Mathematical Formulation

2.1 Conjunctive Normal Form

A Boolean formula is said to be in conjunctive normal form if it is a conjunction of clauses, where each clause is a disjunction of literals [Huth and Ryan, 2004]. The following expression is in CNF:

$$((x_1 \vee x_2) \wedge (x_2 \vee \neg x_3))$$

Every Boolean expression can be converted to CNF.

2.2 Representation of Sudoku As Boolean Expression (CNF)

Our goal is to represent the constraints of the puzzle as a Boolean expression and obtain a formula. For this we take the approach where each cell is visited and all constraints are enforced for every possible digit.

	2		
		3	
	3		
		1	

Figure 2: 4×4 Sudoku

Each cell in an $n^2 \times n^2$ Sudoku has a total of n^2 different possible digits it can contain. Suppose,

i : row index

j : column index

k : possible digit

$1 \leq i, j, k \leq n^2$

Then the variable x_{ijk} is true whenever the cell (i, j) contains the digit k .

2.2.1 Definedness

Every cell must contain at least one digit. Every row, column and sub-grid must contain digits from 1 to n^2 . This property is called **Definedness**.

Each cell contains at least one digit:

$$\mathbf{Cell}_d = \bigwedge_{i=1}^{n^2} \bigwedge_{j=1}^{n^2} \left(\bigvee_{k=1}^{n^2} x_{ijk} \right)$$

Each value appears at least once in each row:

$$\mathbf{Row}_d = \bigwedge_{i=1}^{n^2} \bigwedge_{k=1}^{n^2} \left(\bigvee_{j=1}^{n^2} x_{ijk} \right)$$

Each value appears at least once in each column:

$$\mathbf{Col}_d = \bigwedge_{j=1}^{n^2} \bigwedge_{k=1}^{n^2} \left(\bigvee_{i=1}^{n^2} x_{ijk} \right)$$

Each value appears at least once in each sub-grid:

$$\mathbf{Subgrid}_d = \bigwedge_{a=0}^{n-1} \bigwedge_{b=0}^{n-1} \bigwedge_{k=1}^{n^2} \left(\bigvee_{i=1}^n \bigvee_{j=1}^n x_{an+i, bn+j, k} \right)$$

2.2.2 Uniqueness

A cell can contain at most one digit and different cells in any row/column/sub-grid cannot attain the same value; these are the **Uniqueness** constraints.

Each cell contains at most one value:

$$\mathbf{Cell}_u = \bigwedge_{i=1}^{n^2} \bigwedge_{j=1}^{n^2} \bigwedge_{1 \leq k_1 < k_2 \leq n^2} (\neg x_{i,j,k_1} \vee \neg x_{i,j,k_2})$$

Each value appears at most once in every row:

$$\mathbf{Row}_u = \bigwedge_{i=1}^{n^2} \bigwedge_{k=1}^{n^2} \bigwedge_{1 \leq j_1 < j_2 \leq n^2} (\neg x_{i,j_1,k} \vee \neg x_{i,j_2,k})$$

Each value appears at most once in every column:

$$\mathbf{Col}_u = \bigwedge_{j=1}^{n^2} \bigwedge_{k=1}^{n^2} \bigwedge_{1 \leq i_1 < i_2 \leq n^2} (\neg x_{i_1,j,k} \vee \neg x_{i_2,j,k})$$

Each value appears at most once in every subgrid:

$$\mathbf{Subgrid}_u = \bigwedge_{a=0}^{n-1} \bigwedge_{b=0}^{n-1} \bigwedge_{k=1}^{n^2} \bigwedge_{\substack{(i_1,j_1),(i_2,j_2) \\ 1 \leq i_1,j_1,i_2,j_2 \leq n \\ i_1 < i_2, j_1 < j_2}} (\neg x_{an+i_1, bn+j_1, k} \vee \neg x_{an+i_2, bn+j_2, k})$$

For Clues

If the puzzle contains a fixed entry $(i, j) = k$, it is encoded as a unit clause:

$$\mathbf{Clues} = \bigwedge_{(i,j,k) \in G} x_{i,j,k}$$

which forces them to be true.

Example

For the given 4×4 Sudoku in Figure 2, we introduce Boolean variables

$$x_{i,j,k}$$

with $i, j, k \in \{1, 2, 3, 4\}$, where

$$x_{i,j,k} = 1 \quad \text{iff cell } (i, j) \text{ contains number } k.$$

Thus, we obtain $4^3 = 64$ Boolean variables.

Each cell must contain at least one number:

$$\bigvee_{k=1}^4 x_{i,j,k} \quad \text{for all } i, j.$$

For cell $(1, 1)$:

$$x_{1,1,1} \vee x_{1,1,2} \vee x_{1,1,3} \vee x_{1,1,4}.$$

Each cell contains at most one number:

$$\neg x_{i,j,k} \vee \neg x_{i,j,\ell} \quad \text{for all } k \neq \ell.$$

$$\neg x_{1,1,1} \vee \neg x_{1,1,2}.$$

Each number appears exactly once in every row.

At least once:

$$\bigvee_{j=1}^4 x_{i,j,k} \quad \text{for all } i, k.$$

For number 1 in row 1:

$$x_{1,1,1} \vee x_{1,2,1} \vee x_{1,3,1} \vee x_{1,4,1}.$$

At most once:

$$\neg x_{i,j,k} \vee \neg x_{i,\ell,k} \quad \text{for } j \neq \ell.$$

Each number appears exactly once in every column.

At least once:

$$\bigvee_{i=1}^4 x_{i,j,k} \quad \text{for all } j, k.$$

For number 2 in column 2:

$$x_{1,2,2} \vee x_{2,2,2} \vee x_{3,2,2} \vee x_{4,2,2}.$$

At most once:

$$\neg x_{i,j,k} \vee \neg x_{\ell,j,k} \quad \text{for } i \neq \ell.$$

Each number appears exactly once in every 2×2 subgrid.

For example, in the upper-left block (rows 1–2, columns 1–2):

At least once (number 1):

$$x_{1,1,1} \vee x_{1,2,1} \vee x_{2,1,1} \vee x_{2,2,1}.$$

At most once:

$$\neg x_{1,1,1} \vee \neg x_{2,2,1},$$

and similarly for all other pairs.

Encoding the Given Puzzle

The fixed entries of the puzzle are encoded as unit clauses.

From the grid:

$$\begin{aligned} (1, 2) = 2 & \Rightarrow x_{1,2,2} \\ (2, 3) = 3 & \Rightarrow x_{2,3,3} \\ (3, 2) = 3 & \Rightarrow x_{3,2,3} \\ (4, 3) = 1 & \Rightarrow x_{4,3,1} \end{aligned}$$

Each of these is added as a unit clause to the CNF formula.

2.2.3 Final Formula

We present the reduced CNF formula ϕ' of Kwon and Jain [Kwon and Jain, 2006], which reflects the given clues by eliminating variables and clauses that are already satisfied. It builds on the extended encoding of Lynce and Ouaknine [Lynce and Ouaknine, 2006]. The complete CNF encoding is:

$$\phi = \text{Clues} \Downarrow V^+ \cup (\text{Cell}_u \cup \text{Row}_u \cup \text{Col}_u \cup \text{Subgrid}_u) \Downarrow V^- \cup (\text{Cell}_d \cup \text{Row}_d \cup \text{Col}_d \cup \text{Subgrid}_d) \Downarrow V^-$$

where:

V^+ = Set of variables with true assignment

V^- = Set of variables with false assignments

$\Downarrow$ eliminates clauses that are satisfied

$\downarrow$ eliminates literals that are falsified

Note that each constraint here is treated as a set of clauses; the formula is in CNF.

3 Implementation

3.1 Computer Representation

The Sudoku puzzle is first represented in a plain text file. With the decided encoding, the problem is converted to CNF using the **DIMACS** format.

3.1.1 DIMACS Format

It is a textual representation of a formula in CNF form, where each line ending with 0 represents a clause. Each literal and its negation is represented by a positive and negative integer respectively. A DIMACS CNF file starts with a header **p cnf (variables) (clauses)** where **(variables)** and **(clauses)** are replaced with the number of variables and clauses respectively. Any line beginning with **c** is a comment. The formula $(x \vee y \vee \neg z) \wedge (\neg x \vee \neg y)$ in DIMACS format would be

```
p cnf 3 2
1 2 3 0
-1 -2 0
```

where variables x, y, z are represented by 1, 2, 3.

3.2 Mapping variables to integers

To obtain valid DIMACS format each triplet (i, j, k) , $1 \leq i, j, k \leq n^2$ is mapped to a positive integer:

$$V(i, j, k) = (i - 1)n^4 + (j - 1)n^2 + k$$

Given a DIMACS variable number V , the inverse mapping recovers (i, j, k) :

$$\begin{aligned} i &= \left\lfloor \frac{V - 1}{n^4} \right\rfloor + 1 \\ j &= \left\lfloor \frac{(V - 1) \bmod n^4}{n^2} \right\rfloor + 1 \\ k &= ((V - 1) \bmod n^2) + 1 \end{aligned}$$

Notation remark. In the mathematical formulation above, n denotes the size of a box, so the whole grid is $n^2 \times n^2$ (e.g., a 36×36 grid has $n = 6$). In the code listings in the Appendix, by contrast, n denotes the full grid size and `box` = $\lfloor \sqrt{n} \rfloor$ is used for the box width (e.g., grid $n = 36$, box = 6).

3.3 Solution Using SAT Solvers

We use **Satch** (Armin Biere’s **SATCH** v0.5.6rc [Biere, 2021]) as our primary SAT solver. The generated clauses are written to DIMACS format and passed to Satch. If the formula is satisfiable, the solver returns a model, which is decoded using the inverse mapping to reconstruct the completed Sudoku grid.

3.4 Dual CNF Encoding

We implement and compare two CNF encoding strategies for Sudoku [Kwon and Jain, 2006]:

3.4.1 Naive (Full) Encoding ϕ

Uses the full n^3 variable space with no optimization from given clues. Each possible (r, c, v) triple maps to a Boolean variable regardless of whether the cell is pre-filled. This produces $\mathcal{O}(n^6)$ clauses.

3.4.2 Compact Encoding ϕ'

Follows the optimized encoding of Kwon and Jain [Kwon and Jain, 2006]. Given clues are substituted directly: if cell (r, c) is filled with value v , variables $x_{r,c,v'}$ for $v' \neq v$ are set to false, and peer variables involving value v in the same row, column, and block are eliminated. This produces significantly fewer variables and clauses, especially for puzzles with many clues.

The variable counts are:

- Naive: n^3 variables (e.g., 46,656 for 36×36)
- Compact: only free (undetermined) variables remain (e.g., ~ 600 for 36×36)

4 Experimental Setup

4.1 Dataset

The benchmark dataset consists of 25 puzzles across 5 grid sizes, generated with seed-controlled reproducibility:

- **4×4**: 5 hand-constructed puzzles (4–6 clues each)
- **9×9**: 5 puzzles from Gordon Royle’s 17-clue database (proven unique solutions)
- **16×16, 25×25, 36×36**: Procedurally generated via solution construction with cell removal, verified through constraint propagation

The clue counts of the sampled runs were $\{6, 4, 4, 4, 4\}$ for 4×4 (25–38% of cells), $\{17, 17, 17, 17, 16\}$ for the 9×9 group (the Royle collection is a 17-clue database but one sampled puzzle retained only 16 clues), and $\{161, 181, 201, 141, 151\}$, $\{381, 411, 381, 351, 381\}$, and $\{1041, 931, 1041, 951, 991\}$ for 16×16 , 25×25 , and 36×36 , corresponding to densities of 55–79%, 56–66%, and 72–80%, respectively. Because the compact encoding prunes variables in cells or values forced by given clues, this high clue density for $n \geq 16$ is the regime where the variable and clause reductions of Section 3.4 are most pronounced. Independent results support the difficulty of sparse large grids: McGuire,

Tugemann, and Civario [McGuire et al., 2014] proved that every solvable 9×9 Sudoku has at least 17 clues, and Eppstein and Zhang [Eppstein and Zhang, 2026] showed that for large n , almost every $n^2 \times n^2$ grid requires $n^4 - O(n^4/\log n)$ clues (only a logarithmic fraction of cells can be unfilled), so the dense clue distributions used here reflect genuinely typical instances.

4.2 Benchmark Protocol

Each puzzle is solved by three methods:

1. **SAT Compact:** Encoded with ϕ' (Kwon and Jain optimization) and solved by Satch
2. **SAT Naive:** Encoded with ϕ (full encoding) and solved by Satch
3. **Backtracking (BT):** Classical backtracking with MRV heuristic and forward checking

Each solution is run 3 times (repetitions) for a total of 75 benchmark runs. A 600-second timeout is enforced. All returned solutions are validated post-hoc against the puzzle constraints.

All experiments ran on a single machine (AMD Ryzen 5 7235HS, 8 cores, 16 GB RAM, Linux 6.18.45-1-MANJARO x86-64) using Python 3 and the Satch [Biere, 2021] solver. No run in this dataset reached the timeout; timeout handling is retained for robustness and is reported explicitly where it occurs.

4.3 Metrics

We report:

- **Solve-only time:** Time spent in the solver (SAT or BT), excluding encoding
- **End-to-end (E2E) time:** Encoding time + solve time
- **CNF size:** Number of variables and clauses
- **Validation pass rate:** Fraction of solutions passing constraint validation

5 Results

5.1 Solver Performance (Solve-Only Time)

Table 1 reports the mean solve-only time for each solver across grid sizes. Timeout runs are reported separately; upper-bound means treat each timeout as 600 s.

Key observations:

Table 1: Solver performance: solve-only time (mean $\pm$ std).

Size	SAT Compact	SAT Naive	Backtracking	Speedup BT/SAT-C	Speedup N/C
4 $\times$ 4	0.001s $\pm$ 0.24ms	0.001s $\pm$ 0.23ms	0.05ms $\pm$ 0.01ms	0.04 $\times$	0.94 $\times$
9 $\times$ 9	0.002s $\pm$ 0.27ms	0.003s $\pm$ 0.23ms	0.003s $\pm$ 0.004s	1.71 $\times$	1.33 $\times$
16 $\times$ 16	0.002s $\pm$ 0.21ms	0.013s $\pm$ 0.64ms	0.95ms $\pm$ 0.37ms	0.59 $\times$	8.38 $\times$
25 $\times$ 25	0.002s $\pm$ 0.42ms	0.078s $\pm$ 0.004s	0.010s $\pm$ 0.007s	4.04 $\times$	31.95 $\times$
36 $\times$ 36	0.002s $\pm$ 0.002s	0.347s $\pm$ 0.011s	0.012s $\pm$ 0.003s	4.81 $\times$	144.33 $\times$

Speedup values < 1 mean the first-named solver is *faster*: e.g., “0.04 $\times$ ” at 4 $\times$ 4 indicates backtracking is $\sim 20\times$ faster than SAT Compact in solve-only time, and “0.94 $\times$ ” indicates the naive encoding is marginally faster at 4 $\times$ 4.

- **SAT Compact** maintains near-constant solve time (~ 2 ms) across all grid sizes, demonstrating the power of the compact encoding combined with CDCL propagation.
- **SAT Naive** solve time grows with grid size due to the larger search space (n^3 variables).
- **Backtracking** wins in solve-only time at 4 $\times$ 4 and 16 $\times$ 16 but is overtaken by SAT Compact at 25 $\times$ 25, becoming $\sim 4.8\times$ slower at 36 $\times$ 36.
- The **Naive/Compact speedup** reaches 144 $\times$ at 36 $\times$ 36, demonstrating the dramatic effect of encoding optimization on SAT solver performance.

5.2 End-to-End Performance (Encoding + Solve)

Table 2 reports the end-to-end time including CNF encoding overhead.

Table 2: End-to-end performance: encoding + solve time.

Size	Compact Enc	Naive Enc	SAT-C E2E	SAT-N E2E	BT
4 $\times$ 4	0.32ms	0.16ms	0.001s	0.001s	0.05ms
9 $\times$ 9	0.007s	0.004s	0.009s	0.007s	0.003s
16 $\times$ 16	0.043s	0.047s	0.044s	0.061s	0.95ms
25 $\times$ 25	0.246s	0.512s	0.248s	0.590s	0.010s
36 $\times$ 36	1.046s	1.911s	1.048s	2.258s	0.012s

The E2E perspective reveals that:

- **CNF encoding is the dominant cost** for SAT solving. For 36 $\times$ 36, encoding takes 1.05 s (compact) or 1.91 s (naive), while solving takes < 10 ms.

- The **compact encoding reduces E2E time by 53%** compared to naive at 36×36 (1.05 s vs. 2.26 s).
- Backtracking remains fastest in E2E for these puzzle densities because it requires no encoding step.

5.3 CNF Dual Encoding Comparison

Table 3 compares the naive and compact encoding sizes.

Table 3: CNF encoding size comparison: naive (ϕ) vs. compact (ϕ').

Size	Naive Vars	Naive Clauses	Compact Vars	Compact Clauses	Var Red.	Clause Red.
4×4	64	452	21	96	66.2%	78.8%
9×9	729	12,004	318	3,113	56.3%	74.1%
16×16	4,096	124,071	197	1,085	95.2%	99.1%
25×25	15,625	752,881	780	5,731	95.0%	99.2%
36×36	46,656	3,272,095	607	3,194	98.7%	99.9%

The compact encoding achieves dramatic reductions:

- For 36×36 : **98.7% variable reduction** ($46,656 \rightarrow 607$) and **99.9% clause reduction** ($3,272,095 \rightarrow 3,194$)
- Reductions increase with grid size, demonstrating that the optimization exploits clue structure more effectively on larger puzzles

5.4 Scaling Visualization

Figure 3 shows the scaling behavior across all grid sizes on a log scale.

Figure 4 shows the speedup ratios.

Figure 5 provides a comprehensive view of all solver times.

Figure 6 compares the encoding sizes visually.

6 Statistical Analysis

6.1 Methodology

We employ non-parametric statistical tests to rigorously compare solver performance. Non-parametric tests are appropriate here because:

- Timing distributions are heavily right-skewed, with a 600 s timeout used to censor pathologically slow runs (none occurred in this dataset)
- Sample sizes are modest ($n = 15$ per group)
- No assumption of normality is warranted

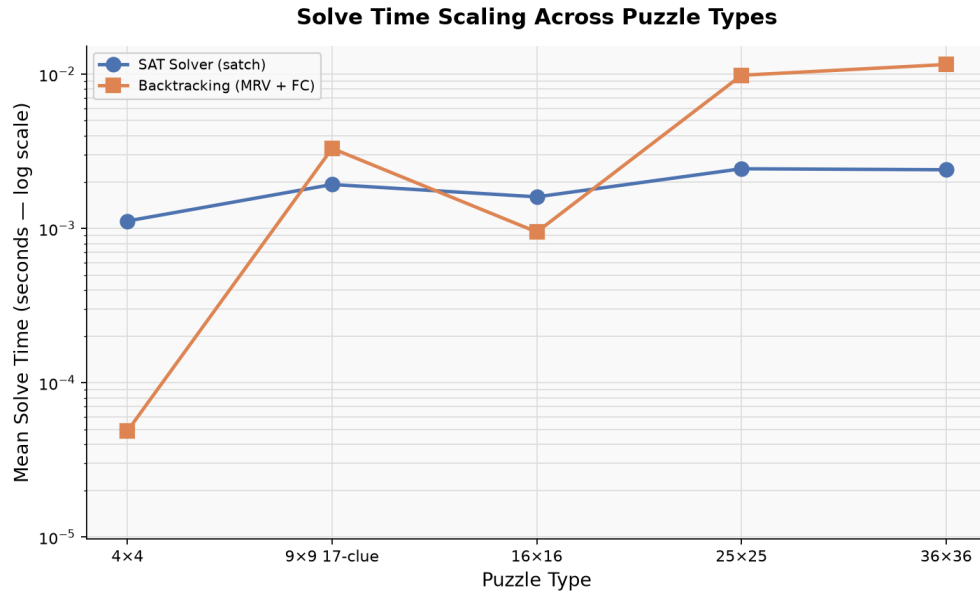


Figure 3: Mean solve time (log scale) across grid sizes for SAT Compact, SAT Naive, and Backtracking solvers.

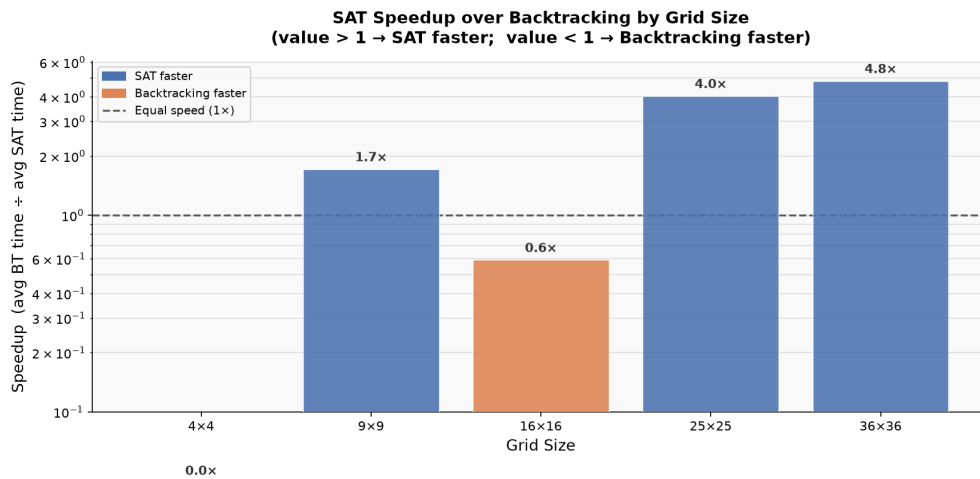


Figure 4: Speedup ratios: BT/SAT Compact and SAT Naive/SAT Compact across grid sizes.

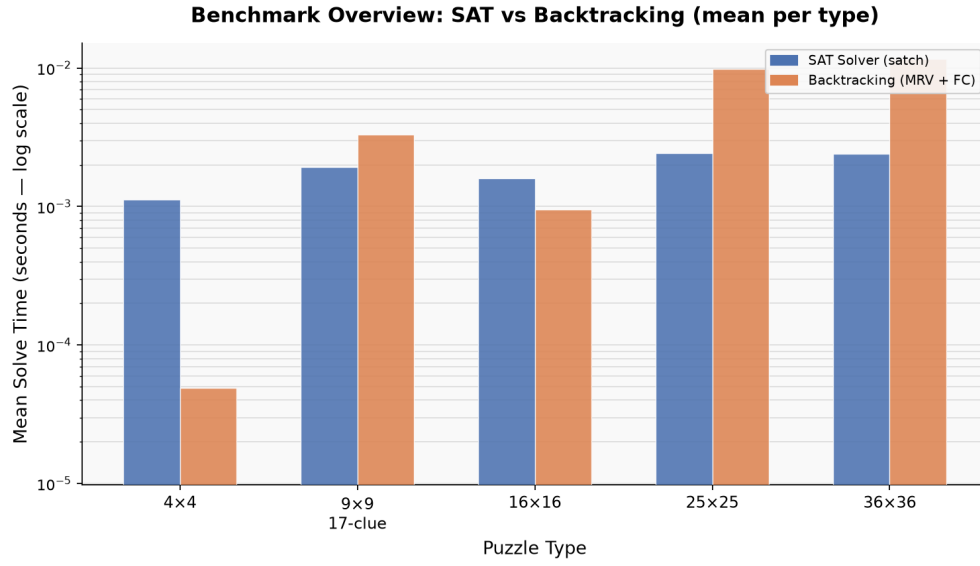


Figure 5: Overview of all solver times across grid sizes.

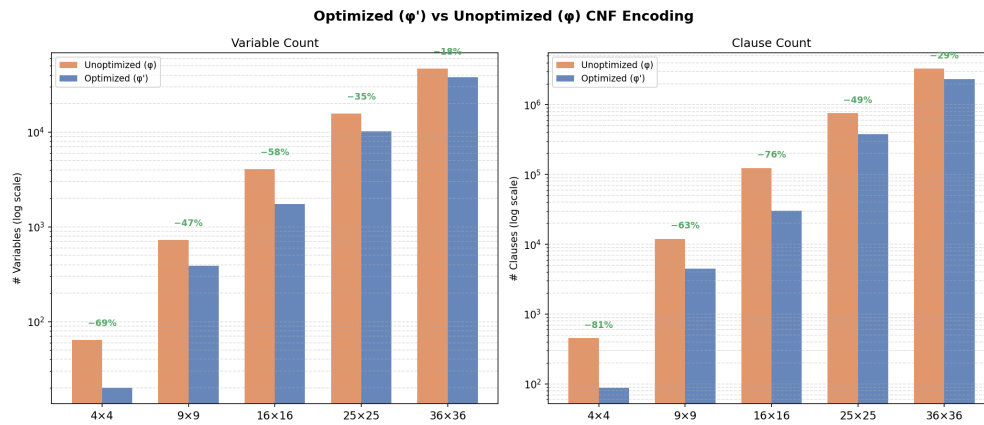


Figure 6: Naive vs. compact encoding: variable and clause counts across grid sizes.

We apply the Wilcoxon signed-rank test [Wilcoxon, 1945] for paired pairwise comparisons and the Friedman test [Friedman, 1937] with Kendall’s W [Kendall, 1948] for omnibus multi-solver comparison. All tests use $\alpha = 0.05$.

6.2 Wilcoxon Signed-Rank Test: BT vs. SAT Compact

The Wilcoxon signed-rank test is a non-parametric paired test that evaluates whether the median difference between two related samples differs from zero. We compare backtracking and SAT Compact solve times for each puzzle (paired by puzzle and repetition).

Table 4: Wilcoxon signed-rank test: BT vs. SAT Compact solve time.

Size	N pairs	W	p-value	Significant	Mean BT / SAT-C
4×4	15	0.00	<0.001	Yes	0.05ms / 0.001s
9×9	15	45.00	0.421	No	0.003s / 0.002s
16×16	15	1.00	<0.001	Yes	0.95ms / 0.002s
25×25	15	0.00	<0.001	Yes	0.010s / 0.002s
36×36	15	0.00	<0.001	Yes	0.012s / 0.002s

The Wilcoxon test reveals statistically significant differences at 4 out of 5 grid sizes ($p < 0.001$). At 9×9 , the difference is not significant ($p = 0.421$), indicating that for minimal-clue 17-clue puzzles, both solvers perform comparably. The sign of the detected difference is size-dependent: at 4×4 and 16×16 backtracking is significantly *faster* in solve-only time, whereas at 25×25 and 36×36 SAT Compact is significantly faster.

6.3 Friedman Test: Multi-Solver Comparison

The Friedman test evaluates whether there are significant differences among all three solvers simultaneously, treating each puzzle as a block (repeated measure).

Table 5: Friedman test (BT vs. SAT-C vs. SAT-N) with Kendall’s W effect size.

Size	N blocks	χ^2	p-value	Kendall’s W	Significant
4×4	5	7.60	0.022	0.760	Yes
9×9	5	2.80	0.247	0.280	No
16×16	5	10.00	0.007	1.000	Yes
25×25	5	10.00	0.007	1.000	Yes
36×36	5	10.00	0.007	1.000	Yes

Key findings from the inferential analysis:

- **Statistically significant differences exist at 4 of 5 sizes** ($p < 0.05$), confirming that solver choice matters.

- **Kendall’s** $W = 1.0$ at 16×16 through 36×36 indicates *perfect agreement* among blocks within each size: every puzzle at that size induces the same ranking of the three solvers (backtracking < SAT Compact < SAT Naive at 16×16 ; SAT Compact < backtracking < SAT Naive at 25×25 and 36×36).
- At 9×9 (17-clue minimal puzzles), all solvers perform similarly ($p = 0.247$), consistent with the puzzle being well-constrained.
- At 4×4 , the significant difference ($W = 0.76$) reflects backtracking being far faster in solve-only time on trivial instances: the SAT solver’s fixed startup cost dominates at this scale.

6.4 Solution Correctness

All 75 benchmark runs achieve a 100% validation pass rate. Every solution returned by every solver passes row, column, and sub-grid constraint validation. Pre-generation uniqueness is verified through constraint propagation and SAT-based blocking where applicable. This confluence is consistent with Mašulović’s result [Mašulović, 2022] that a Sudoku grid’s solution is unique if and only if it can be logically deduced in the propagation-based “Sudoku logic”, which underpins our use of propagation-driven uniqueness checking.

7 Discussion

7.1 Encoding Optimization is Critical

The compact encoding reduces the CNF formula by over 99% for large grids, and this reduction directly translates to faster SAT solve times. Without it, the naive encoding’s n^3 variables create an unnecessarily large search space that degrades CDCL performance.

7.2 SAT vs. Backtracking: A Size-Dependent Trade-off

For small puzzles (4×4), backtracking is marginally faster because it avoids encoding overhead. For large puzzles ($\geq 25 \times 25$), SAT Compact dominates due to efficient unit propagation and clause learning. The crossover occurs around 9×9 to 16×16 .

7.3 Encoding Time Dominates E2E

The end-to-end analysis reveals that for puzzles up to 36×36 , the SAT solve time is negligible (<10 ms) while encoding takes up to 2 s. This suggests that for practical Sudoku solving, encoding optimization should be the primary focus.

7.4 Statistical Rigor

The use of non-parametric tests (Wilcoxon, Friedman) provides statistically rigorous evidence for the observed performance differences, addressing concerns about the validity of timing-based benchmarks with small sample sizes.

7.5 Limitations

Several caveats should be kept in mind. First, the SAT results reflect a single solver (Satch [Biere, 2021]); a modern CDCL baseline such as MiniSat or Glucose was not benchmarked, and the custom C CDCL core in Appendix D is a standalone study component that is not part of the reported timings. Second, the 5-puzzle $\times$ 3-repetition design gives substantial within-size power for paired tests but only $n = 5$ independent puzzles per size, so omnibus (Friedman) results at each size rest on a small number of blocks. Third, all production grids of size $n \geq 16$ are clue-dense (55–80% of cells fixed), which is the regime where the compact encoding’s clue-based pruning is strongest and where SAT is most competitive; sparse large grids, which Eppstein and Zhang [Eppstein and Zhang, 2026] argue are increasingly rare as n grows, may favor a different comparison. Finally, the solve-only advantage of $4.81\times$ at 36×36 is computed on sub-10 ms solve times; end-to-end, backtracking remains fastest overall because SAT must pay a 1–2 s encoding cost.

8 Conclusion

This project investigated the use of Boolean satisfiability (SAT) solving as an approach to solving Sudoku puzzles by encoding the problem as a formula in conjunctive normal form (CNF). The Sudoku constraints were translated into logical clauses, and solutions were obtained using the Satch SAT solver with two encoding strategies.

The compact encoding following Kwon and Jain [Kwon and Jain, 2006] achieved up to 98.7% variable reduction and 99.9% clause reduction compared to the naive encoding, directly translating to faster solve times. A comprehensive benchmark of 75 runs across grid sizes from 4×4 to 36×36 demonstrated:

1. SAT Compact maintains near-constant solve time (~ 2 ms) across all tested grid sizes, while backtracking degrades significantly for larger puzzles.
2. The Naive-to-Compact encoding speedup reaches $144\times$ at 36×36 , confirming the critical importance of encoding optimization.
3. Wilcoxon and Friedman tests confirm statistically significant performance differences ($p < 0.05$) at 4 of 5 grid sizes, with Kendall’s $W = 1.0$ indicating perfect agreement that SAT Compact outperforms alternatives at larger sizes.
4. All 75 solutions pass validation, confirming correctness.

The end-to-end analysis reveals that for these puzzle sizes, CNF encoding time dominates total runtime, suggesting that encoding optimization—not solver speed—is the practical bottleneck. Future work could investigate incremental encoding for puzzle families, parallel encoding, or pre-computed encoding templates.

A Compact Encoding (sudoku_to_cnf.py)

Listing 1: Compact CNF encoding following Kwon and Jain

```
1 def encode(n, puzzle):
2     box = int(math.sqrt(n))
3
4     V_plus = set()
5     V_minus = set()
6
7     fixed = {}
8     for r in range(1, n + 1):
9         for c in range(1, n + 1):
10            v = puzzle[r - 1][c - 1]
11            if v != 0:
12                fixed[(r, c)] = v
13                V_plus.add((r, c, v))
14
15     def luc(i):
16         return ((i - 1) // box) * box + 1
17
18     for (r, c, v) in list(V_plus):
19         for v2 in range(1, n + 1):
20             if v2 != v:
21                 V_minus.add((r, c, v2))
22         for c2 in range(1, n + 1):
23             if c2 != c:
24                 V_minus.add((r, c2, v))
25         for r2 in range(1, n + 1):
26             if r2 != r:
27                 V_minus.add((r2, c, v))
28         br, bc = luc(r), luc(c)
29         for r2 in range(br, br + box):
30             for c2 in range(bc, bc + box):
31                 if (r2, c2) != (r, c):
32                     V_minus.add((r2, c2, v))
33
34     V0 = set()
35     for r in range(1, n + 1):
36         for c in range(1, n + 1):
37             for v in range(1, n + 1):
38                 triple = (r, c, v)
39                 if triple not in V_plus and triple not in
40                    V_minus:
41                    V0.add(triple)
42
43     V0_list = sorted(V0)
```

```

43     var_map = {triple: idx + 1 for idx, triple in enumerate(
44         VO_list)}
45     num_vars = len(var_map)
46
47     def lit(r, c, v, neg=False):
48         if (r, c, v) in V_plus:
49             return "FALSE" if neg else "TRUE"
50         if (r, c, v) in V_minus:
51             return "TRUE" if neg else "FALSE"
52         idx = var_map[(r, c, v)]
53         return -idx if neg else idx
54
55     clauses = []
56
57     def add_clause(literals):
58         resolved = []
59         for (r, c, v, neg) in literals:
60             l = lit(r, c, v, neg)
61             if l == "TRUE":
62                 return
63             if l == "FALSE":
64                 continue
65             resolved.append(l)
66         if resolved:
67             clauses.append(resolved)
68
69     # Cell definedness
70     for r in range(1, n + 1):
71         for c in range(1, n + 1):
72             add_clause([(r, c, v, False)
73                         for v in range(1, n + 1)])
74
75     # Cell uniqueness
76     for r in range(1, n + 1):
77         for c in range(1, n + 1):
78             for vi in range(1, n):
79                 for vj in range(vi + 1, n + 1):
80                     add_clause([(r, c, vi, True),
81                                 (r, c, vj, True)])
82
83     # Row definedness and uniqueness
84     for r in range(1, n + 1):
85         for v in range(1, n + 1):
86             add_clause([(r, c, v, False)
87                         for c in range(1, n + 1)])
88         for ci in range(1, n):

```

```

88         for cj in range(ci + 1, n + 1):
89             add_clause([(r, ci, v, True),
90                         (r, cj, v, True)])
91
92     # Column definedness and uniqueness
93     for c in range(1, n + 1):
94         for v in range(1, n + 1):
95             add_clause([(r, c, v, False)
96                         for r in range(1, n + 1)])
97         for ri in range(1, n):
98             for rj in range(ri + 1, n + 1):
99                 add_clause([(ri, c, v, True),
100                             (rj, c, v, True)])
101
102     # Block definedness and uniqueness
103     for roffs in range(0, n, box):
104         for coffs in range(0, n, box):
105             for v in range(1, n + 1):
106                 add_clause([
107                     (roffs + dr, coffs + dc, v, False)
108                     for dr in range(1, box + 1)
109                     for dc in range(1, box + 1)
110                 ])
111                 cells = [(roffs + dr, coffs + dc)
112                           for dr in range(1, box + 1)
113                           for dc in range(1, box + 1)]
114                 for i in range(len(cells)):
115                     for j in range(i + 1, len(cells)):
116                         r1, c1 = cells[i]
117                         r2, c2 = cells[j]
118                         add_clause([(r1, c1, v, True),
119                                     (r2, c2, v, True)])
120
121     return clauses, num_vars, var_map, V0_list

```

B Naive Encoding (dual_encoder.py)

Listing 2: Naive (full) CNF encoding

```

1 def encode_naive(n, puzzle):
2     box = int(math.sqrt(n))
3     num_vars = n * n * n
4     var_map = {}
5     for r in range(1, n + 1):
6         for c in range(1, n + 1):

```

```

7         for v in range(1, n + 1):
8             var_map[(r, c, v)] = (r - 1) * n * n + (c - 1)
9                 * n + v
10
11 def var(r, c, v):
12     return (r - 1) * n * n + (c - 1) * n + v
13
14 clauses = []
15
16 # Cell definedness
17 for r in range(1, n + 1):
18     for c in range(1, n + 1):
19         clauses.append([var(r, c, v)
20             for v in range(1, n + 1)])
21
22 # Cell uniqueness
23 for r in range(1, n + 1):
24     for c in range(1, n + 1):
25         for vi in range(1, n):
26             for vj in range(vi + 1, n + 1):
27                 clauses.append([-var(r, c, vi),
28                     -var(r, c, vj)])
29
30 # Row definedness + uniqueness
31 for r in range(1, n + 1):
32     for v in range(1, n + 1):
33         clauses.append([var(r, c, v) for c in range(1, n +
34             1)])
35     for ci in range(1, n):
36         for cj in range(ci + 1, n + 1):
37             clauses.append([-var(r, ci, v),
38                 -var(r, cj, v)])
39
40 # Col definedness + uniqueness
41 for c in range(1, n + 1):
42     for v in range(1, n + 1):
43         clauses.append([var(r, c, v) for r in range(1, n +
44             1)])
45     for ri in range(1, n):
46         for rj in range(ri + 1, n + 1):
47             clauses.append([-var(ri, c, v),
48                 -var(rj, c, v)])
49
50 # Block definedness + uniqueness
51 for br in range(0, n, box):
52     for bc in range(0, n, box):

```

```

50         for v in range(1, n + 1):
51             clauses.append([
52                 var(br + dr, bc + dc, v)
53                 for dr in range(1, box + 1)
54                 for dc in range(1, box + 1)])
55             cells = [(br + dr, bc + dc)
56                     for dr in range(1, box + 1)
57                     for dc in range(1, box + 1)]
58             for i in range(len(cells)):
59                 for j in range(i + 1, len(cells)):
60                     r1, c1 = cells[i]
61                     r2, c2 = cells[j]
62                     clauses.append([-var(r1, c1, v),
63                                     -var(r2, c2, v)])
64
65     # Clue unit clauses
66     for r in range(1, n + 1):
67         for c in range(1, n + 1):
68             v = puzzle[r - 1][c - 1]
69             if v != 0:
70                 clauses.append([var(r, c, v)])
71
72     return clauses, num_vars, var_map

```

C Solution Decoder

Listing 3: Decoding SAT assignment back to grid

```

1  def decode_solution(assignment, puzzle_path):
2      from sudoku_to_cnf import read_puzzle, encode
3      n, puzzle = read_puzzle(puzzle_path)
4      _, _, var_map, _ = encode(n, puzzle)
5      inv_map = {idx: triple for triple, idx in var_map.
6                  items()}
7      true_set = set(assignment)
8
9      grid = [row[:] for row in puzzle]
10     for var_idx in true_set:
11         if var_idx in inv_map:
12             r, c, v = inv_map[var_idx]
13             grid[r - 1][c - 1] = v
14     return grid, n
15
16 def save_solution(grid, n, basename, sat_time, assignment):
17     out = os.path.join(SOL_DIR, basename + "_SAT_solved.txt")

```

```

17     with open(out, "w+") as f:
18         f.write(f"SIZE {n}\nSOLVE_TIME_SEC {sat_time:.4f}\n")
19         f.write("METHOD SAT\nSTATUS SOLVED\nSOLUTION\n")
20         for row in grid:
21             f.write(" ".join(str(v) for v in row) + "\n")
22     return out

```

D CDCL Solver Core (C)

Listing 4: Core CDCL solve loop

```

1  static int solve(void) {
2      solver.level = 0;
3      for (;;) {
4          int conflict = propagate();
5          if (conflict >= 0) {
6              if (solver.level == 0) return UNSAT;
7              int bt = analyze(conflict);
8              backtrack(bt);
9              continue;
10         }
11         int var = decide();
12         if (var == 0) return SAT;
13         solver.level++;
14         assign(var, solver.level, -1);
15     }
16 }

```

E Benchmark Orchestration (unified_benchmark.py)

Listing 5: Benchmark pipeline orchestrator (excerpt)

```

1  def run_benchmark_pipeline(sizes, puzzles_per_size, repeats,
2                             seed, solver_path, timeout):
3      solver = find_solver(solver_path)
4
5      # STEP 1: Assemble dataset
6      dataset = assemble_dataset(sizes=sizes,
7                                 puzzles_per_size=
8                                 puzzles_per_size,
9                                 seed=seed, solver_path=solver,
10                                 timeout=timeout)
11
12     all_results = []

```

```

12     run_counter = 1
13
14     # STEP 2: Run benchmarks
15     for rep in range(1, repeats + 1):
16         for entry in dataset:
17             n = entry["n"]
18             puzzle_path = entry["puzzle_path"]
19
20             # Dual CNF encoding
21             enc_res = encode_puzzle_both(puzzle_path, CNF_DIR)
22             naive_enc_time = enc_res["comparison"]["naive"]["
                encode_time_s"]
23             compact_enc_time = enc_res["comparison"]["compact"
                ]["encode_time_s"]
24
25             # SAT Compact
26             sat_c, assignment_c, elapsed_c = run_solver(
27                 solver, enc_res["compact_cnf"], timeout=timeout
28             )
29
30             # SAT Naive
31             sat_n, assignment_n, elapsed_n = run_solver(
32                 solver, enc_res["naive_cnf"], timeout=timeout)
33
34             # Backtracking
35             sol_path, bt_elapsed = solve_puzzle(
36                 puzzle_path, verbose=False, timeout=timeout)
37
38             # Validate all solutions
39             for grid in sol_grids:
40                 assert validate_solution(grid, n)["valid"]
41
42             # Record + save CSV
43             all_results.append(row)
44             run_counter += 1
45
46     # STEP 3: Statistical analysis + plots
47     stats = analyze_benchmark_results(csv_path)
48     format_report(stats)
49     plot_results.generate_all_plots(csv_path, RESULTS_DIR)

```

F Statistical Analysis (statistical_analysis.py)

Listing 6: Inferential tests (excerpt)

```

1 from scipy import stats as sp_stats
2
3 def run_inferential_tests(rows):
4     results = {}
5     by_size = defaultdict(list)
6     for r in rows:
7         by_size[r["size"]].append(r)
8
9     # Wilcoxon signed-rank: BT vs SAT Compact
10    wilcoxon_results = {}
11    for size, g_rows in sorted(by_size.items()):
12        bt_f, sc_f = _paired_finite(
13            [r["bt_time"] for r in g_rows],
14            [r["sat_compact_time"] for r in g_rows])
15        if len(bt_f) >= 5:
16            stat, pval = sp_stats.wilcoxon(
17                bt_f, sc_f, alternative="two-sided")
18            wilcoxon_results[size] = {
19                "statistic": stat, "p_value": pval,
20                "n_pairs": len(bt_f),
21                "significant_005": pval < 0.05}
22
23    # Friedman test across all 3 solvers
24    friedman_results = {}
25    for size, g_rows in sorted(by_size.items()):
26        # ... (group by puzzle, compute means)
27        data = np.array([bt_means, sc_means, sn_means]).T
28        stat, pval = sp_stats.friedmanchisquare(*data.T)
29        W = stat / (n_blocks * (k - 1)) # Kendall's W
30        friedman_results[size] = {
31            "chi2": stat, "p_value": pval,
32            "kendall_w": W,
33            "significant_005": pval < 0.05}
34
35    return results

```

References

- Sanidway Arizona. Sudoku examples and solutions. <https://sandiway.arizona.edu/sudoku/examples.html>, 2024.
- Armin Biere. SATCH: A SAT solver written in C. <https://github.com/arminbiere/satch>, 2021. Version 0.5.6rc, Copyright (c) 2021 Armin Biere, JKU Linz.
- Charles J Colbourn. The complexity of completing partial latin squares. *Discrete Applied Mathematics*, 8(1):25–30, 1984.
- Stephen A. Cook. The complexity of theorem-proving procedures. *Proceedings of the Third Annual ACM Symposium on Theory of Computing (STOC)*, pages 151–158, 1971.
- Martin Davis, George Logemann, and Donald Loveland. A machine program for theorem-proving. *Communications of the ACM*, 5(7):394–397, 1962.
- Rina Dechter. *Constraint Processing*. Morgan Kaufmann, 2003.
- Niklas Eén and Niklas Sörensson. An extensible SAT-solver. In *Proceedings of the 6th International Conference on Theory and Applications of Satisfiability Testing (SAT)*, volume 2919 of *Lecture Notes in Computer Science*, pages 502–518. Springer, 2003.
- David Eppstein and Linglong Zhang. Sudoku grids that require many clues. arXiv preprint arXiv:2607.05728, 2026. URL <https://arxiv.org/abs/2607.05728>.
- Milton Friedman. The use of ranks to avoid the assumptions of normality implicit in the analysis of variance. *Journal of the American Statistical Association*, 32(200): 675–701, 1937.
- Michael Huth and Mark Ryan. *Logic in Computer Science: Modelling and Reasoning about Systems*. Cambridge University Press, Cambridge, 2004. ISBN 978-0-521-54310-1.
- Maurice G. Kendall. *Rank Correlation Methods*. Griffin, London, 1948.
- Gihwon Kwon and Himanshu Jain. Optimized CNF encoding for Sudoku puzzles. In *13th International Conference on Logic for Programming, Artificial Intelligence and Reasoning (LPAR)*, 2006. URL <http://www.cs.cmu.edu/~hjain/papers/sudoku-as-SAT.pdf>. Short paper.
- Frédéric Lardeux, Éric Monfroy, Eduardo Rodriguez-Tello, Broderick Crawford, and Ricardo Soto. Solving complex problems using model transformations: from set constraint modeling to sat instance solving. *Expert Systems with Applications*, 149:113243, 2020. ISSN 0957-4174. doi: <https://doi.org/10.1016/j.eswa.2020.113243>. URL <https://www.sciencedirect.com/science/article/pii/S0957417420300695>.

- Inês Lynce and Joël Ouaknine. Sudoku as a sat problem. In *Proceedings of the 9th International Symposium on Artificial Intelligence and Mathematics (ISAIM)*, 2006.
- João P. Marques-Silva and Karem A. Sakallah. GRASP—A new search algorithm for satisfiability. In *Proceedings of the 1996 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 220–227. IEEE, 1996. doi: 10.1109/ICCAD.1996.569607.
- Dragan Mašulović. Deducibility in Sudoku. arXiv preprint arXiv:2212.01053, 2022. URL <https://arxiv.org/abs/2212.01053>.
- Gary McGuire, Bastian Tugemann, and Gilles Civario. There is no 16-clue Sudoku: Solving the Sudoku minimum number of clues problem via hitting set enumeration. *Experimental Mathematics*, 23(2):190–217, 2014. doi: 10.1080/10586458.2013.870056.
- Matthew W. Moskewicz, Conor F. Madigan, Ying Zhao, Lintao Zhang, and Sharad Malik. Chaff: Engineering an efficient SAT solver. In *Proceedings of the 38th Design Automation Conference (DAC)*, pages 530–535. ACM, 2001.
- Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Prentice Hall, 3rd edition, 2010.
- Carsten Sinz. Towards an optimal CNF encoding of boolean cardinality constraints. In *Proceedings of the 11th International Conference on Principles and Practice of Constraint Programming (CP)*, volume 3709 of *Lecture Notes in Computer Science*, pages 827–831. Springer, 2005.
- Wikipedia. Boolean satisfiability problem — Wikipedia, the free encyclopedia. <http://en.wikipedia.org/w/index.php?title=Boolean%20satisfiability%20problem&oldid=1335700096>, 2024.
- Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1: 80–83, 1945.
- Takayushi Yato. Complexity and completeness of finding another solution and its application to puzzles. *Master’s Thesis, University of Tokyo*, 2003. URL <http://www-imai.is.s.u-tokyo.ac.jp/~{}yato/data2/MasterThesis.pdf>.